

Step 4 — A Timeout and a Partial-Result Fallback — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# exceptions.py
class PipelineStepFailedError(Exception):
    pass
```

```
# pipeline.py (only the changed run_pipeline shown)
from exceptions import PipelineStepFailedError

def run_pipeline(topic, max_rounds, progress_sink=None):
    feedback = None
    rounds = []
    draft = None
    verdict = None

    for round_number in range(1, max_rounds + 1):
        try:
            draft = run_researcher(topic, feedback=feedback)
            verdict = run_fact_checker(draft)
        except Exception as exc:
            raise PipelineStepFailedError(f"Round {round_number} failed: {exc}")

        round_info = RevisionRound(
            round_number=round_number, summary=draft.summary,
            approved=verdict.approved, issues=verdict.issues,
        )
        rounds.append(round_info)
        if progress_sink is not None:
            progress_sink(round_info)

    if verdict.approved:
```

```

        break
    feedback = "; ".join(verdict.issues)

    return PipelineResult(
        topic=topic, final_summary=draft.summary, approved=verdict.approved,
        rounds=rounds, revision_count=len(rounds),
    )

```

```

# mcp_server.py (only the changed deep_research shown)
import asyncio

@mcp.tool()
async def deep_research(topic):
    progress_log = []

    def record_round(round_info):
        progress_log.append(round_info)

    try:
        result = await asyncio.wait_for(
            asyncio.to_thread(run_pipeline, topic, 3, record_round),
            timeout=60,
        )
    except asyncio.TimeoutError:
        return "TIMED OUT. Partial rounds: " + str(progress_log)
    except PipelineStepFailedError as exc:
        return f"FAILED: {exc}. Partial rounds: {progress_log}"

    return format_result(result)

```

This works and covers both failure paths. The fallback messages are ugly (a raw Python list printed with `str()`) and the timeout/round-count are hardcoded instead of read from config — both worth fixing next.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the full, typed, config-driven version

```
# config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv

load_dotenv()

@dataclass
class Config:
    log_level: str
    openai_api_key: str
    max_revision_rounds: int
    timeout_seconds: int

def load_config() -> Config:
    return Config(
        log_level=os.getenv("LOG_LEVEL", "INFO"),
        openai_api_key=os.getenv("OPENAI_API_KEY", ""),
        max_revision_rounds=int(os.getenv("MAX_REVISION_ROUNDS", "3")),
        timeout_seconds=int(os.getenv("DEEP_RESEARCH_TIMEOUT_SECONDS", "60")),
    )
```

```
# exceptions.py
class PipelineStepFailedError(Exception):
    """Raised when the Researcher or Fact-Checker fails unexpectedly
    inside a pipeline round -- carries which round and why."""
```

```
# pipeline.py
from pydantic import BaseModel
from typing import Callable, Optional

from researcher import run_researcher, Draft
from fact_checker import run_fact_checker, FactCheckVerdict
```

```

from exceptions import PipelineStepFailedError

class RevisionRound(BaseModel):
    round_number: int
    summary: str
    approved: bool
    issues: list[str]

class PipelineResult(BaseModel):
    topic: str
    final_summary: str
    approved: bool
    rounds: list[RevisionRound]
    revision_count: int

def run_pipeline(
    topic: str,
    max_rounds: int,
    progress_sink: Optional[Callable[[RevisionRound], None]] = None,
) -> PipelineResult:
    feedback: str | None = None
    rounds: list[RevisionRound] = []
    draft: Draft | None = None
    verdict: FactCheckVerdict | None = None

    for round_number in range(1, max_rounds + 1):
        try:
            draft = run_researcher(topic, feedback=feedback)
            verdict = run_fact_checker(draft)
        except Exception as exc:
            raise PipelineStepFailedError(
                f"Round {round_number} failed: {exc}"
            ) from exc

        round_info = RevisionRound(
            round_number=round_number,
            summary=draft.summary,
            approved=verdict.approved,
            issues=verdict.issues,
        )
        rounds.append(round_info)
    if progress_sink is not None:
        progress_sink(round_info)

```

```

        if verdict.approved:
            break
        feedback = "; ".join(verdict.issues)

    return PipelineResult(
        topic=topic,
        final_summary=draft.summary,
        approved=verdict.approved,
        rounds=rounds,
        revision_count=len(rounds),
    )

```

```

# mcp_server.py
import asyncio

from mcp.server.fastmcp import FastMCP

from pipeline import run_pipeline, PipelineResult, RevisionRound
from exceptions import PipelineStepFailedError
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

mcp = FastMCP("deep-research-service")

def format_result(result: PipelineResult) -> str:
    lines = [f"Deep research on: {result.topic}"]
    if not result.approved:
        lines.append("[NOT FULLY CONFIRMED -- fact-check cap reached before approval]")
        lines.append("")
        lines.append("Revision trace:")
        for round_info in result.rounds:
            status = "approved" if round_info.approved else "rejected"
            lines.append(f"  Round {round_info.round_number}: {status}")
            for issue in round_info.issues:
                lines.append(f"    - {issue}")
        lines.append("")
    if result.approved:
        lines.append(f"Status: approved after {result.revision_count} round(s).")
    else:
        lines.append(f"Status: NOT approved -- hit the {result.revision_count}-round limit.")
    lines.append("")
    lines.append("Summary:")

```

```

lines.append(result.final_summary)
return "\n".join(lines)

def format_partial_result(topic: str, rounds: list[RevisionRound], note: str) ->
    lines = [
        f"Deep research on: {topic}",
        "[UNCONFIRMED -- this did not finish normally]",
        note,
        "",
    ]
    if rounds:
        lines.append("Revision trace so far:")
        for round_info in rounds:
            status = "approved" if round_info.approved else "rejected"
            lines.append(f" Round {round_info.round_number}: {status}")
        lines.append("")
        lines.append("Best draft available (NOT confirmed as final):")
        lines.append(rounds[-1].summary)
    else:
        lines.append("No draft was produced before this happened.")
    return "\n".join(lines)

@mcp.tool()
async def deep_research(topic: str) -> str:
    """Research a topic through a Researcher/Fact-Checker loop and return a
    fact-checked summary with a short revision trace. This can take several
    seconds. If it can't finish in time, or a step fails, it returns the
    best draft found so far, clearly marked as unconfirmed -- never a hang,
    never a crash, never an empty result."""
    progress_log: list[RevisionRound] = []

    def record_round(round_info: RevisionRound) -> None:
        progress_log.append(round_info)
        logger.info(
            "Round %d: %s", round_info.round_number,
            "approved" if round_info.approved else "rejected",
        )

    logger.info("deep_research called for topic=%r", topic)
    try:
        result = await asyncio.wait_for(
            asyncio.to_thread(
                run_pipeline, topic, config.max_revision_rounds, record_round
            ),
            timeout=config.timeout_seconds,

```

```

    )
except asyncio.TimeoutError:
    logger.warning(
        "deep_research timed out after %ds for topic=%r",
        config.timeout_seconds, topic,
    )
    return format_partial_result(
        topic, progress_log,
        f"Timed out after {config.timeout_seconds} seconds.",
    )
except PipelineStepFailedError as exc:
    logger.error("deep_research pipeline step failed: %s", exc)
    return format_partial_result(
        topic, progress_log, f"A pipeline step failed: {exc}"
    )

logger.info(
    "deep_research finished for topic=%r: approved=%s, rounds=%d",
    topic, result.approved, result.revision_count,
)
return format_result(result)

if __name__ == "__main__":
    logger.info("Starting deep-research-service over stdio")
    mcp.run(transport="stdio")

```

Expected behavior: - Normal topic, normal speed → same output as Step 3, unchanged. - Artificially slow topic (see Advanced below for how to force this) → after `DEEP_RESEARCH_TIMEOUT_SECONDS`, returns a string starting with `[UNCONFIRMED -- this did not finish normally]`, showing whatever rounds completed before the timeout and the best draft among them. - Forced exception inside `run_researcher` → returns a string starting with `[UNCONFIRMED -- this did not finish normally]` naming which round failed and why, instead of an unhandled crash.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — forcing both unhappy paths on purpose, to actually prove the fallback works

```
# test_step4_unhappy_paths.py -- a throwaway script, run once by hand
import time
from unittest.mock import patch

import mcp_server
import researcher

def slow_researcher(topic, feedback=None):
    time.sleep(120) # much longer than DEEP_RESEARCH_TIMEOUT_SECONDS
    return researcher.Draft(topic=topic, summary="never gets here")

def broken_researcher(topic, feedback=None):
    raise RuntimeError("pretend network failure")

async def check_timeout():
    with patch("pipeline.run_researcher", side_effect=slow_researcher):
        result = await mcp_server.deep_research("a slow topic")
        assert "[UNCONFIRMED]" in result, "expected an unconfirmed fallback on timeout"
        print("Timeout path OK:\n", result[:200], "...")

async def check_step_failure():
    with patch("pipeline.run_researcher", side_effect=broken_researcher):
        result = await mcp_server.deep_research("a broken topic")
        assert "[UNCONFIRMED]" in result, "expected an unconfirmed fallback on failure"
        assert "pretend network failure" in result, "expected the real error to be na"
        print("Failure path OK:\n", result[:200], "...")

import asyncio
asyncio.run(check_timeout())
asyncio.run(check_step_failure())
```

Expected output:

```

Timeout path OK:
  Deep research on: a slow topic
[UNCONFIRMED -- this did not finish normally]
Timed out after 60 seconds.
...

Failure path OK:
  Deep research on: a broken topic
[UNCONFIRMED -- this did not finish normally]
A pipeline step failed: Round 1 failed: pretend network failure
...

```

This is the actual proof both unhappy paths work — not by reading the `try / except` and trusting it, but by forcing exactly the two failure modes Step 4 promises to handle and checking the result names them clearly. Notice `check_timeout()` still takes about `DEEP_RESEARCH_TIMEOUT_SECONDS` real seconds to run, even though it "fails fast" from the caller's point of view — the orphaned thread is still sleeping for 120 seconds in the background after the test moves on, exactly the limitation the Advanced hint described.

Approach 2 — a short, honest note about the orphaned-thread limitation, kept next to the code

```

# mcp_server.py -- add this comment directly above deep_research(), so the
# limitation is documented exactly where a future reader would need it:

# NOTE on timeouts: asyncio.wait_for() stops US from waiting past
# config.timeout_seconds -- it does NOT stop run_pipeline() itself, which
# keeps running in its background thread until it finishes or errors on its
# own. This is a known, accepted limitation: Python can't safely force-kill
# a thread. A production version needing TRUE cancellation would run the
# pipeline in a separate process (which can be killed) or add a
# should_stop() check inside pipeline.py's loop. Not needed for this
# project's scope.

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate is the complete, correct implementation. Approach 1 is proof — deliberately forcing both failure modes and asserting on the real result, instead of hoping the code is right. Approach 2 adds no behavior at all; it writes the one limitation worth knowing directly into the file where a future reader (including future-you) would otherwise have to rediscover it the hard way.

Which one should you actually write? Intermediate Approach 1 is what belongs in your final `config.py`, `exceptions.py`, `pipeline.py`, and `mcp_server.py`. Run Advanced Approach 1's

forced-failure test at least once by hand — it's the only way to be sure the fallback path genuinely works, rather than just compiles. Keep Approach 2's comment; a two-line note that saves a confusing debugging session later is one of the cheapest good habits in this whole project.